

Pattern Problems

A Cheatsheet

The Core Process

Every pattern problem follows the same 4 steps, in order:

- Draw it out — write what each row looks like on paper
- Count manually — for each part (spaces, stars, values), note the count at each row
- Spot the change — how does the count change as i increases?
- Express as formula — the loop limit or printed value follows directly

The code is always the last step, never the first.

Reading Loop Counts

For any loop starting at 0 with step 1, the count equals the upper bound directly:

`for (int j = 0; j < LIMIT; j++)` → runs exactly `LIMIT` times

Loop	Count
<code>j < n</code>	<code>n</code>
<code>j < n - i</code>	<code>n - i</code>
<code>j < 2*i + 1</code>	<code>2i + 1</code>
<code>j <= n</code>	<code>n + 1</code>
<code>j < n - i - 1</code>	<code>n - i - 1</code>

Deriving the Formula

Write out the count at each row and ask: what is the starting value, and how does it change?

Row (i)	Count	Change	Formula
0, 1, 2, 3, 4	1, 2, 3, 4, 5	+1 each row	$i + 1$
0, 1, 2, 3, 4	4, 3, 2, 1, 0	-1 each row	$N - i - 1$
0, 1, 2, 3, 4	1, 3, 5, 7, 9	+2 each row	$2*i + 1$

When Count Becomes a Condition

Some patterns don't vary by count — they vary by what gets printed at a given (i, j). Ask: what coordinates satisfy this cell being '*' or a certain value?

Condition	When to use it
$i == 0 \parallel i == n-1 \parallel j == 0 \parallel j == n-1$	Border/hollow rectangle
$(i + j) \% 2 == 0$	Alternating 0/1 pattern
$\min(i, j, 2n-2-i, 2n-2-j)$	Distance from nearest edge (concentric layers)
N - distance	Value at a given layer

Formula for the Printed Value

When the loop increments from 0 but you need a decreasing value, use a formula instead of decrementing the loop:

Want: $i+1, i, i-1, \dots, 1$ at positions $j=0, 1, 2, \dots$

Value: $i + 1 - j$

General rule: starting value minus j. Loop structure stays the same; formula does the work.

Compound & Complex Patterns

Multi-part rows

Break the row into parts and handle each with its own loop:

spaces loop | stars/values loop | trailing loop

Each loop's limit comes from its own formula, derived independently.

Symmetric patterns (diamond, hourglass)

Split into two halves — derive each half separately, then stack the loops.

Concentric layer patterns

When value changes in rings (like an onion), use distance from nearest edge:

```
distance = Math.min(i, j, (2*N-2)-i, (2*N-2)-j)
value     = N - distance
```

Signal: If the border has one value, next layer has another, and so on inward — this is the approach.

ASCII / Alphabet patterns

Map j to a character using 'A' as an offset:

```
(char)('A' + j)           // A, B, C, D ...  
(char)('A' + i + 1 - j)  // descending with formula
```

Tip: Use 'A' directly in code instead of the raw number 65 — makes intent obvious.

The Mindset

- Code is always the last step. Pattern first, formula second, code third.
- Start from 0 and increment. Use a formula for the value if needed.
- Complex pattern = multiple simple parts. Break it down.
- When stuck on conditions, ask: what coordinates share this property?
- Trust your instincts when clear. Fall back to the formula when not.